Unit-1

- Version Control System - git
- Continuous Integration Tool Jenkin

Collaborative Environment ??

What is “version control”

and why should you care?

A very Popular Tool : Google Sheets ??

- Problems ?
- Accidental overwrites : History is not granular; hard to track specific cell authorship or changes when many edits are made
- Data deletion :Only helps if you act fast or know exactly when the change happened
- Multiple copies : Can still be messy if people download or create offline edits
- Blame / audit trail : Not detailed per-cell change log; hard to track bulk edits or understand intent
- Collaboration : Great for discussion, but lacks structured review and approval flow like Git’s pull requests

Another Popular Tool : Microsoft Word Online ??

Git - Basics

What is “version control”

and why should you care?

- Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.
- Save and manage different versions of your files and code , keep track of changes, and undo mistakes AND work with others.
- Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. So ideally, we can place any file in the computer on version control.

- **What is Git ?**
- Git is a free and open-source version control system, originally created by Linus Torvalds in 2005. Unlike older centralized version control systems such as SVN and CVS, Git is distributed: every developer has the full history of their code repository locally.
- It has excellent support for branching, merging, and rewriting repository history, which has led to many innovative and powerful workflows and tools.
- It allows teams to collaborate on Git branches and efficiently review each other’s code.

Git - Commands

Basic Git Commands

1. git config
2. git init
3. git clone
4. git add
5. git commit
6. git diff
7. git reset
8. git status
9. git rm
10. git log
11. git show
12. git tag
13. git branch
14. git checkout
15. git merge
10. git remote
11. git push
16. git pull
17. git stash
18. …….. Many More

1. Working Directory (Untracked or Modified)
   - This is actual project folder where you're editing files.
     - **Untracked** (Git doesn't know about them yet)
     - **Modified** (Git sees they've changed)
   - Git doesn't automatically track changes—you must tell it what to track
2. Staging (Index)
   - We can decide what we want to commit using: git add <file>
   - We can stage multiple files or everything at once: git add .
3. Repository (Committed)
   - Saves the current version **permanently in the Git history**

Git - Commands (contd …)

Step-1 : Create a working directory of your choice. Change the folder and check if your folder is indeed created.

MINGW64:/d/IGDTU/Aug-Dec(2025)/gIT/Praveen

```
DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT
$ mkdir /d/IGDTU/Aug-Dec\(2025\)/gIT/Praveen

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT
$ cd /d/IGDTU/Aug-Dec\(2025\)/gIT/Praveen

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen
$ ls

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen
$
```

What git init does:

- The ***git init*** command is a fundamental Git command used to create a new Git repository. This creates a hidden .git subdirectory within that folder, which is where Git stores all the necessary metadata and objects for managing the repository's history and configuration.
- It creates the .git directory, which is the core of local Git repository. This directory contains essential components like:

Git - Commands (contd …)

- ❑ HEAD: A reference to the current branch.
- ❑ config: Repository-specific configuration settings.
- ❑ objects: Stores all the actual data (blobs, trees, commits) that make up project's history.
- ❑ refs: Stores references to branches and tags.

- It sets up the basic structure for Git to begin tracking changes within that directory.
- Running ***git init*** in a directory that already contains a .git repository is safe; it will reinitialize the repository without overwriting existing data.
- git init only creates the repository structure; it does not automatically add or commit any existing files. After running git init, we will typically use ***git add*** to stage files and ***git commit*** to save the initial version of project.

Step-

MINGW64:/d/IGDTU/Aug-Dec(2025)/gIT/Praveen

```
DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen
$ git config --global init.defaultBranch Trunk_Development

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen
$ git config --global --get init.defaultBranch
Trunk_Development
```

Git - Commands (contd …)

Step-3 : Now Initialise the repository

```
MINGW64:/d/IGDTU/Aug-Dec(2025)/gIT/Praveen

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen
$ git init
Initialized empty Git repository in D:/IGDTU/Aug-Dec(2025)/gIT/Praveen/.git/
```

What git add does:

- The ***git add*** command is a fundamental Git command used to add changes from the working directory to the staging area (also known as the index). This command is the first step in recording changes to project's history in Git.
- It tells Git that we want to include specific changes (new files, modifications to existing files, or deletions) in the next commit.
- However, ***git add*** doesn't really affect the repository in any significant way i.e changes are not actually recorded until we run ***git commit***.
- First, we edit files in the working directory. When they are ready to save a we stage changes with ***git add.*** If we are happy with the staged snapshot, we proceed to commit it to the project history with ***git commit***

Git - Commands (contd …)

What git commit does:

- The git commit command is used in Git to save a snapshot of the changes in the staging area to the local repository.
- It operates on the changes that have been explicitly added to the staging area using ***git add***. Only staged changes are included in the commit.
- ***git commit*** only saves changes to the local Git repository. To share these commits with a remote repository (like GitHub or GitLab), a subsequent ***git push*** command is required.

Difference between git commit and git push ?

- ***git commit*** and ***git push*** serve completely different roles. When we run ***git commit,*** Git takes the changes that are already staged using git add and records them as a new snapshot in local repository. This operation affects only local machine; it does not require an internet connection and does not send anything to GitHub, GitLab. It can be reversed.
- In contrast, ***git push*** is a network operation whose job is to transfer locally created commits to a remote repository. Only when we push the changes become visible to collaborators or backed up on the remote server. Until that moment, commits exist solely in the local .git database. It is irreversible.
- ***git commit*** is generally reversible, while ***git push*** is not easily

Tracking changes in git

What git log does:

- To view the change history of a particular file, use the ***git log*** command. It lists all commits made to a repository in reverse chronological order.
- The listing for each commit includes the the commit’s author, when it was created, and the log message Git was given when the commit was created.
- If we need see everything in one line then we can use command

$ ***git log –oneline***

- To see now content from my second version ,

git show

```
DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen (Trunk_Development)
$ git log
commit 8e0d6ed98ab43369963ef50c0efe228dee8ce1a8 (HEAD -> Trunk_Development)
Author: praveenprithvi705 <67621976+praveenprithvi705@users.noreply.github.com>
Date:   Thu Aug 21 12:27:34 2025 +0530

    Third revision - After Manager Approval

commit f9b7b3eb6eabd98e0732b6a78d19bc57e10837a3
Author: praveenprithvi705 <67621976+praveenprithvi705@users.noreply.github.com>
Date:   Thu Aug 21 12:24:59 2025 +0530

    Second Revision after Tech Lead comments incorporated

commit e37d7744d7349cad8910aa985d19410e868ddba8
Author: praveenprithvi705 <67621976+praveenprithvi705@users.noreply.github.com>
Date:   Thu Aug 21 12:15:57 2025 +0530

     This is after Code review done on 15-AUG-2025

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen (Trunk_Development)
$
```

```
DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen (Trunk_Development)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen (Trunk_Development)
$ git commit -m "Third revision - After Manager Approval"
[Trunk_Development 8e0d6ed] Third revision - After Manager Approval
 1 file changed, 2 insertions(+), 1 deletion(-)

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen (Trunk_Development)
$ git log --oneline --graph --decorate --all
* 8e0d6ed (HEAD -> Trunk_Development) Third revision - After Manager Approval
* f9b7b3e Second Revision after Tech Lead comments incorporated
* e37d774  This is after Code review done on 15-AUG-2025

DELL@DESKTOP-LL366IO MINGW64 /d/IGDTU/Aug-Dec(2025)/gIT/Praveen (Trunk_Development)
$
```

Git branching Strategies

Git or any other version control tools allow developers to track, manage and organize their code. It allow developers to diverge from the main branch by creating separate branches to isolate or manage their code changes. The default branch in Git is the master or main branch. Thus effective branch management is crucial for successful collaboration and efficient development with Git.

Common Git branching strategies:

1. Trunk-based development

Trunk-based development is a branching strategy that in fact requires no branches but instead, developers integrate their changes into a shared trunk at least once a day. This shared trunk should be ready for release anytime. The main idea behind this strategy is that developers make smaller changes more frequently and thus the goal is to limit long-lasting branches and avoid merge conflicts as all developers work on the same branch. In other words, developers commit directly into the trunk without the use of branches.

Consequently, trunk-based development is a key enabler of continuous integration (CI) and continuous delivery (CD) since changes are done more frequently to the trunk, often multiple times a day (CI) which allows features to be released much faster (CD).

However the con is that it requires an experienced team that can slice features appropriately for regular integration. It also makes mandatory

Git branching Strategies(contd …)

2. GitFlow

GitFlow enables parallel development where developers can work separately from the master branch on features where a feature branch is created from the master branch. When changes are complete, the developer merges these changes back to the master branch for release. This branching strategy consists of the following branches:

- Master
- Develop
- Feature- to develop new features that branches off the develop branch
- Release- help prepare a new production release; usually branched from the develop branch and must be merged back to both develop and master
- Hotfix- also helps prepare for a release but unlike release branches, hotfix branches arise from a bug that has been discovered and must be resolved; it enables developers to keep working on their own changes on the develop branch while the bug is being fixed.

Consequently , GitFlow is well-suited for large teams and aligning work across multiple teams. Effective handling of multiple product versions. Clear responsibilities for each branch and it allows for easy navigation of production versions through tags.

The cons of this strategy are that it is a little complex due to numerous branches, potentially leading to merge conflicts. It may cause development and release frequency slow due to multi-step process. Also, it requires team consensus and commitment to adhere to the strategy.

Git branching Strategies(contd …)

3. GitHubFlow

GitHub-Flow removes the problems in Git-Flow by eliminating release branches. It start off with the main branch where developers create feature branches that stem directly from the Master or Main branch, to isolate their work which are then merged back . Features and bug fixes are implemented using long-living feature branches. The feature branch is then deleted once work is complete. The main idea behind this approach is keeping the master code in a constant deployable state and hence can support continuous integration and continuous delivery processes.

- Master: The GitHub Flow starts with the Master/Main branch, which contains the most recent stable code ready for release.
- Feature: Developers initiate feature branches from the main branch to implement new features or address bugs. Upon completion, the feature branch is merged back into the main branch. If a merge conflict arises, developers are required to resolve it prior to finalizing the merge.

The pros in this strategy are , since there is no Development branch as you are testing and we are going to delete feature branch after merge the bug history / changes are lost. This strategy is particularly suited for small teams and web applications and it is ideal when we need to maintain a single production version and code is of high / good quality. The con is that the lack of development branches makes this strategy more susceptible to bugs and so can lead to an unstable production code if branches are not properly tested before merging with the master-release. A poor quality work may cause

GitHub (contd …)

3. GitLabFlow

GitLab Flow is a simpler alternative to GitFlow & GitHubFlow that combines feature-driven development and feature branching with issue tracking. GitLab Flow introduces a intermediate staging environment separate from the production environment to address bug fixings. Whenever the main branch is ready to be deployed the fixed code is merged back into the production branch and release it. Thus it has

- Master: Main production branch housing stable release ready code.
- Feature/Development : Developers initiate this branch from the Main/Master branch to implement new features or address bugs.
- Release/Staging: Prior to a Production, a release branch is created from the develop branch. This branch is used to combine new features and bug fixes for the release after exhaustive testing. If still any bugs found they are fixed on Development branch and same process follows. Upon completion, developers merge the changes from the release branch into both the develop and main branches. Thus this branch can be loosely viewed as staging branch as well.

Thus this strategy ,offers a robust and scalable branching strategy suitable for small and even larger teams and projects. It ensures a distinct separation between code under development and production-ready code, minimizing the risk of accidental changes to the production code.

The con is GitLab Flow may slow down the process as it calls for merging changes to develop branch. It may pose challenges due to its complexity,

Continuous Integration Tools - Jenkin

Continuous Integration (CI) :

- Continuous Integration is about how developers integrate code using a shared repository multiple times a day with the help of automation. It is a practice of merging frequently - rather than merging in a large change at the end of a development cycle.
- Generally , each integration triggers an automated build and test pipeline that validates the correctness of the change as depicted in figure (next slide)
- In other words , CI enforces discipline by discovering errors surface quickly rather than accumulating so that developed code remains in a releasable state at all times
- This is achieved through automation which is central to CI process and eliminates manual build and testing steps that are error-prone and slow.
- CI pipelines as shown in next slide create a standardized quality gate applied uniformly across contributors. This consistency improves reliability and reduces environment-related surprises and enhances deployment readiness.
- In a DevOps environment , Integration with multiple repositories or microservices increases orchestration complexity. Another difficulty in devops environment for CI is maintaining deterministic builds is difficult due to configuration drift between developer machines and CI environments/pipelines.
- To ensure a CI good health it demands continuous investment in test

Jenkins

- Jenkins is an open source continuous integration (CI) server. It manages and controls several stages of the software delivery process, including build, documentation, automated testing, packaging, and static code analysis.
- Highly popular in DevOps environment.
- Jenkins automation is commonly triggered by code changes in repositories like GitHub, Bitbucket, and GitLab.
- It gets integrated with build tools like Maven and supports the use of container technologies like Docker and Kubernetes for testing and packaging of software releases.
- Jenkins architecture primarily operates in a distributed model, utilizing a Master -Agent configuration to manage and execute build jobs efficiently.

Jenkins Master (Controller):

The Jenkins Master is the central server in a Jenkins setup.

- It is responsible for scheduling build jobs, distributing them to available agents, and monitoring the agents' status (online/offline).
- The Master also collects build results from the agents and presents them through its web-based user interface.
- It can also execute build jobs directly if configured to do so.

Jenkins Agents (Slaves/Nodes):

Jenkins Agents are machines (physical, virtual, or containerized) that

Jenkins(contd)

- Multiple Jenkins Agents can be configured to balance build load, improve performance, and create a secure environment independent of the Controller
- Agents can be configured to provide different execution environments (e.g., Linux, Windows, macOS) to cater to diverse project requirements.
- They require a Java installation and a network connection to the Jenkins Master to communicate and receive instructions.

Communication:

- Communication between the Jenkins Master and Agents typically occurs via TCP/IP.
- The Master sends build instructions to the agents, and the agents return build results and status updates to the Master.
- The Master-Agent architecture benefits as it enables distributed builds, allowing for parallel execution of different parts of a job or multiple jobs simultaneously across various agents.
- This distribution enhances scalability, improves performance for large and complex projects, and allows for testing in diverse environments.

Jenkins(contd)

Jenkin Typical workflow :

- Developers modify the source code, committing changes to the repository, and Jenkins creates a new build in order to handle the new Git commit.
- Jenkins can work in “push” or “pull” mode. The Jenkins CI server is either triggered by an event such as a code commit, or it can regularly check the repository for changes.
- The build server builds the code and generates an artifact. If the build fails, the developer receives an alert.
- Jenkins deploys the built application/executable to the test server, which can execute continuous, automated tests. Developers receive alerts if their changes impact functionality.
- Jenkins optionally deploys the changes to the production server if the code has no issues

Jenkins Build Triggers

Jenkins build triggers are mechanisms that start a build process in Jenkins. Triggers determine when and under what conditions a build should be initiated. This automation is crucial for continuous integration and continuous deployment (CI/CD), as it allows developers to integrate code changes more frequently, identifying issues early in the development cycle. Build triggers in Jenkins cover various scenarios, such as code commits, scheduled times, or external systems initiating a build.

Jenkins(contd)

1. Manual Build Trigger

A manual build trigger in Jenkins allows users to start a build process manually through the Jenkins user interface. This trigger is useful for jobs that require human oversight or intervention, such as builds that need to be run after a thorough code review or jobs that should be executed to test specific features or fixes.

2. Scheduled Build Trigger

Scheduled build triggers in Jenkins enable builds to be started at predefined times using cron-like scheduling syntax. This feature is ideal for tasks that need to run at regular intervals without manual intervention, such as nightly builds, weekly reports, or periodic integration tests.

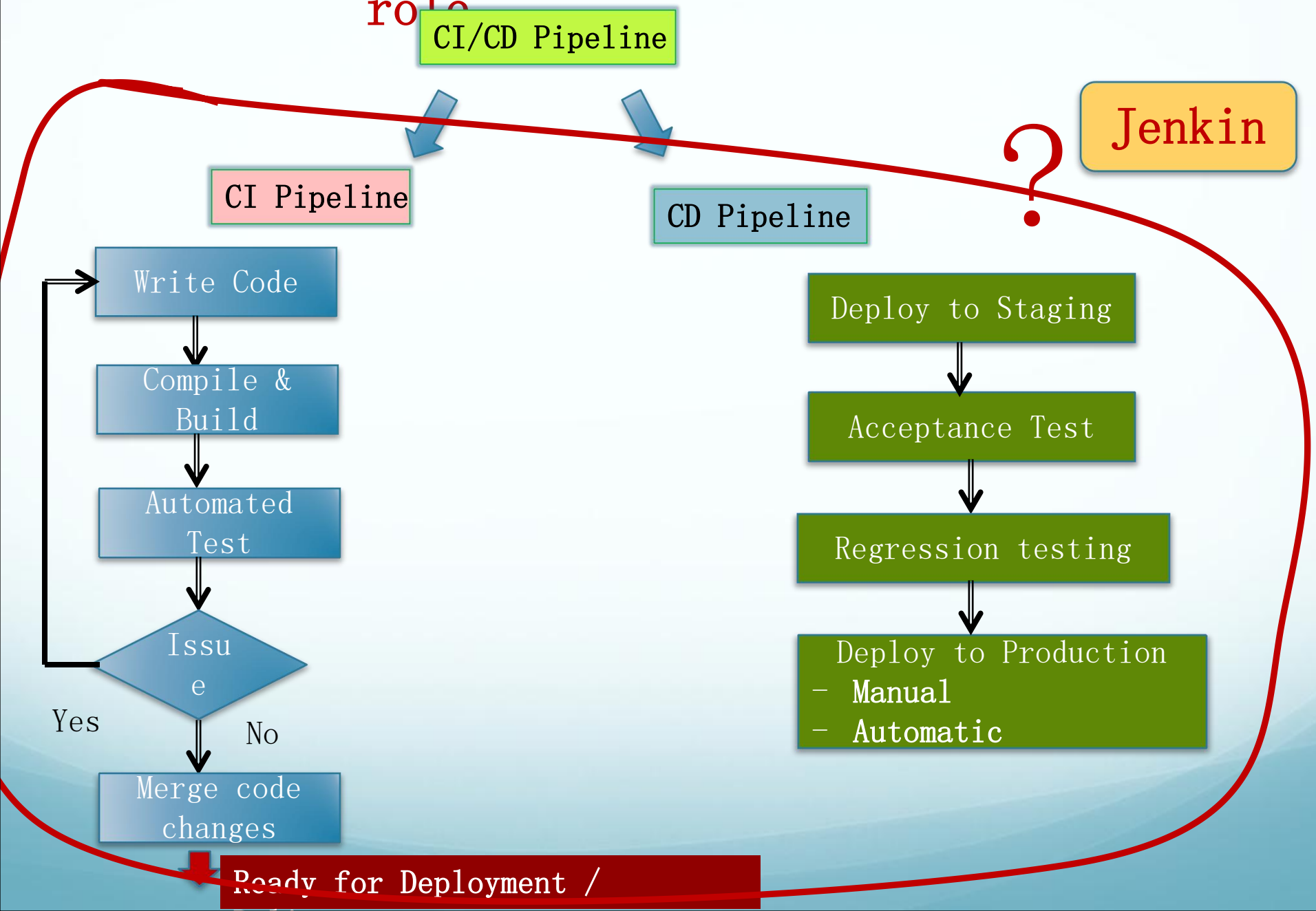
3. SCM (Source Code Management) Trigger

The SCM trigger is one of the most commonly used build triggers in Jenkins, initiating a build whenever changes are detected in the source code repository. This trigger is crucial for continuous integration practices, as it ensures that the latest code changes are automatically tested and integrated into the main codebase.

4. Webhook Trigger

A webhooks trigger allows Jenkins to automatically start a build when a specific event occurs in an external system. This is particularly useful for integrating Jenkins with tools like GitHub or GitLab, where a webhook can be

CI / CD - Jenkin role



CI / CD - Jenkin role (contd)

Jenkins Workflow in CI/CD

- Developer Commit:
  - Developers make code changes and commit these changes to a version control system (e.g., Git). This step initiates the CI/CD pipeline. Committing code changes ensures that they are tracked and versioned properly.
- Trigger:
  - The version control system detects the new commit and triggers the build process automatically. Jenkins detects the commit (via webhook or polling etc). Automated triggering of the build process ensures that new changes are continuously integrated and tested.
- Build:
  - The codebase is compiled and built into a deployable artifact, such as a binary executable or a Docker image. Common tools include Maven, Gradle, etc. The build step verifies that the code compiles correctly and that all dependencies are resolved.
- Notify the Build Outcome:
  - The CI/CD system notifies the team of the build results, whether it passed or failed. Immediate feedback on the build status helps developers quickly identify and resolve any build issues. Notification systems can include emails, chat integrations or dashboards.

CI / CD - Jenkin role (contd)

- Deliver Build to Staging:
  - If tests pass, the build artifact is deployed to a staging environment. The staging environment simulates the production environment, allowing for final validation before production deployment.
- Test:
  - Automated tests are run to verify quality. Automated tests are executed on the build artifact. Testing frameworks like JUnit, Selenium, TestNG, pytest, etc. Running tests ensures that the new code does not introduce any bugs or regressions and that it meets the required quality standards.
- Deploy to Production:
  - After successful validation in staging, the build is automatically or manually promoted to the production environment. This step makes the new features and fixes available to end-users. Deployment strategies might include blue-green deployment, canary releases, or rolling updates to minimize downtime and risk.

Thanks